TIP102 | Intermediate Technical Interview Prep

Intermediate Technical Interview Prep Fall 2026 (@ Section 1 | Tuesdays and Thursdays 3PM - 5PM PT)
Personal Member ID#: **134071**

Need help? Post on our **class slack channel** or email us at **support@codepath.org**

Getting Started

Learning with AI ✨

▶ Course Info

Course Progress

Week 1

Week 2

Week 3

Week 4

Week 5

Week 6

Week 7

Week 8

Week 9

Week 10

Week 11

Week 12

Week 8: Session 2

Binary Trees I

Students continue to work with binary trees and are introduced to foundational algorithms for a common subcategory of trees: binary search trees (BSTs). The problems encourage exploring key operations such as inserting and removing nodes, checking tree balance, finding specific nodes, and traversing trees in various orders.

> You can find session recordings and more on the resources tab. Session slide decks are available on the overview tab.

📈 Part 1 : Instructor Led Session

We'll spend the first portion of the synchronous class time in large groups, where the instructor will lead class instruction for 30-45 minutes.

👩‍💻 Part 2: Breakout Session

In breakout sessions, we will explore and collaboratively solve problem sets in small groups. Here, the **collaboration, conversation, and approach** are just as important as “solving the problem” - please engage warmly, clearly, and plentifully in the process!

In breakout rooms you will:

- Screen-share the problem/s, and verbally review them together
- Screen-share an interactive coding environment, and talk through the steps of a solution approach
    - ProTip: - An Integrated Development Environment (IDE) is a fancy name for a tool you could use for shared writing of code - like VSCode, PyCharm, Replit.com, Collabed.it, CodePen.io, or other - your staff team will specify which tool to use for this class!
- Screen-share an implementation of your proposed solution
- Independently follow-along, or create an implementation, in your own IDE.

Your program leader/s will indicate which code sharing tool/s to use as a group, and will help break down or provide specific scaffolding with the main concepts above.

▶ **Note on Expectations**

🔎 Problem Solving Approach

We will approach problems using the six steps in the UMPIRE approach.

UMPIRE: Understand, Match, Plan, Implement, Review, Evaluate.

We'll apply these six steps to the problems we'll see in the first half of the course.

We will learn to:

- **Understand** the problem
- **Match** identifies common approaches you've seen/used before
- **Plan** a solution step-by-step, and
- **Implement** the solution
- **Review** your solution
- **Evaluate** your solution's time and space complexity and think critically about the advantages and disadvantages of your chosen approach.

ℹ️ Note: Testing your Binary Tree (Printing)

To keep the amount of starter code manageable, we have chosen not to include a function to print a binary tree as part of each relevant problem statement. You may instead copy the function in the drop-down below `print_tree()` and use it as needed while you complete the problem sets.

▶ Print Binary Tree Function

ℹ️ Note: Testing your Binary Tree (Generating a Tree)

Now that you have practice manually building trees for testing in previous sessions, we are providing a function that builds binary trees based off of a list of values to speed up the testing process. We have chosen not to include this function in the starter code for each problem to keep the length of problems manageable. You may instead copy the function in the drop-down below `build_tree()` and use it as needed while you complete the problem sets.

▶ Build Binary Tree Function

Problem Set Version 1

▼ Problem 1: Monstera Madness

Given the `root` of a binary tree where each node represents the number of splits in a leaf of a Monstera plant, return the number of Monstera leaves 🌿 that have an odd number of splits.

Evaluate the time complexity of your function. Define your variables and provide a rationale for why you believe your solution has the stated time complexity.

Note: The term *leaf* in this problem refers to the plant leaf 🌿 of a Monstera plant, not the type of node *leaf nodes* which are nodes with no children.

```
class TreeNode():
     def __init__(self, value, left=None, right=None):
         self.val = value
         self.left = left
         self.right = right

def count_odd_splits(root):
    pass
```

Example Usage:

```
"""
      2
     / \
    /   \
   3     5
  / \     \
 6   7     12
"""

# Using build_tree() function included at top of page
values = [2, 3, 5, 6, 7, None, 12]
monstera = build_tree(values)

print(count_odd_splits(monstera))
print(count_odd_splits(None))
```

Example Output:

```
3
Example 1 Explanation: Three Monstera leaves (nodes) have an odd number of fenestrations (
0
```

▶ ✨ AI Hint: Traversing Trees

Close Section

▼ Problem 2: Flower Finding

You are looking to buy a new flower plant for your garden. The nursery you visit stores its inventory in a binary search tree (BST) where each node represents a plant in the store. The plants are organized according to their names (`val` s) in alphabetical order in the BST.

Given the root of the binary search tree `inventory` and a target flower `name`, write a function `find_flower()` that returns `True` if the flower is present in the garden and `False` otherwise.

Evaluate the time complexity of your function. Define your variables and provide a rationale for why you believe your solution has the stated time complexity. Assume the input tree is balanced when calculating time complexity.

```
class TreeNode():
     def __init__(self, value, left=None, right=None):
         self.val = value
         self.left = left
         self.right = right

def find_flower(inventory, name):
    pass
```

Example Usage:

```
"""
         Rose
        /    \
      Lilac   Tulip
     /  \       \
  Daisy  Lily  Violet
"""

# using build_tree() function at top of page
values = ["Rose", "Lilac", "Tulip", "Daisy", "Lily", None, "Violet"]

garden = build_tree(values)

print(find_flower(garden, "Lilac"))
print(find_flower(garden, "Sunflower"))
```

Example Output:

```
True
False
```

▶ ✨ AI Hint: Binary Search Trees

Close Section

▼ Problem 3: Flower Finding II

Consider the following function `non_bst_find_flower()` which accepts the root of a binary tree `inventory` and a flower `name`, and returns `True` if a flower (node) with `name` exists in the binary tree. Unlike the previous problem, this tree is **not** a binary search tree.

1. Compare your solution to `find_flower()` in Problem 2 to the following solution. Discuss with your group: How is the code different? Why?
2. What is the time complexity of `non_bst_find_flower()`? How does it compare to the time complexity of `find_flower()` in Problem 2?
3. How would the time complexity of `find_flower()` from Problem 2 change if the tree `inventory` was not balanced?

```
class TreeNode:
    def __init__(self, value, left=None, right=None):
        self.val = value
        self.left = left
        self.right = right

def non_bst_find_flower(root, name):
    if root is None:
        return False

    if root.val == name:
        return True

    return non_bst_find_flower(root.left, name) or non_bst_find_flower(root.right, name)
```

Example Usage:

```
"""
         Daisy
        /    \
      Lily   Tulip
     /  \       \
  Rose  Violet  Lilac
"""

# using build_tree() function at top of page
values = ["Rose", "Lily", "Tulip", "Daisy", "Lilac", None, "Violet"]
garden = build_tree(values)

print(non_bst_find_flower(garden, "Lilac"))
print(non_bst_find_flower(garden, "Sunflower"))
```

Example Output:

```
True
False
```

Close Section

▼ Problem 4: Adding a New Plant to the Collection

You have just purchased a new houseplant and are excited to add it to your collection! Your collection is meticulously organized using a Binary Search Tree (BST) where each node in the tree represents a houseplant in your collection, and houseplants are organized alphabetically by name (`val`).

Given the root of your BST `collection` and a new houseplant `name`, insert a new node with value `name` into your collection. Return the root of your updated collection. If another plant with `name` already exists in the tree, add the new node in the existing node's right subtree.

Evaluate the time complexity of your function. Define your variables and provide a rationale for why you believe your solution has the stated time complexity. Assume the input tree is balanced when calculating time complexity.

```
class TreeNode:
    def __init__(self, value, left=None, right=None):
        self.val = value
        self.left = left
        self.right = right

def add_plant(collection, name):
    pass
```

Example Usage:

```
"""
            Money Tree
        /               \
Fiddle Leaf Fig    Snake Plant
"""

# Using build_tree() function at the top of page
values = ["Money Tree", "Fiddle Leaf Fig", "Snake Plant"]
collection = build_tree(values)

# Using print_tree() function at the top of page
print_tree(add_plant(collection, "Aloe"))
```

Example Output:

```
['Money Tree', 'Fiddle Leaf Fig', 'Snake Plant', 'Aloe']

Explanation:
Tree should have the following structure:
           Money Tree
        /               \
 Fiddle Leaf Fig   Snake Plant
```

```
  /
Aloe
```

▶ ✨ AI Hint: Binary Search Trees

Close Section

▼ Problem 5: Sorting Plants by Rarity

You are going to a plant swap where you can exchange cuttings of your plants for new plants from other plant enthusiasts. You want to bring a mix of cuttings from both common and rare plants in your collection. You track your plant collection in a BST where each node has a `key` and a `val`. The `val` contains the plant name, and the `key` is an integer representing the plant's rarity. Plants are organized in the BST by their `key`.

To help choose which plants to bring, write a function `sort_plants()` which takes in the BST root `collection` and returns an array of plant nodes as tuples in the form `(key, val)` sorted from least to most rare. Sorted order can be achieved by performing an **inorder traversal** of the BST.

```
class TreeNode:
    def __init__(self, key, value, left=None, right=None):
        self.key = key      # Plant rarity
        self.val = value      # Plant name
        self.left = left
        self.right = right


def sort_plants(collection):
    pass
```

Example Usage:

```
"""
         (3, "Monstera")
        /               \
   (1, "Pothos")     (5, "Witchcraft Orchid")
        \                 /
  (2, "Spider Plant")   (4, "Hoya Motoskei")
"""

# Using build_tree() function at the top of page
values = [(3, "Monstera"), (1, "Pothos"), (5, "Witchcraft Orchid"), None, (2, "Spider Plar
collection = build_tree(values)

print(sort_plants(collection))
```

Example Output:

```
[(1, 'Pothos'), (2, 'Spider Plant'), (3, 'Monstera'), (4, 'Hoya Motoskei'), (5, 'Witchcraf
```

Close Section

▼ Problem 6: Finding a New Plant Within Budget

You are looking for a new plant and have a max `budget`. The plant store that you are shopping at stores their inventory in a BST where each node has a `key` representing the price of the plant and `value` contains the plant's name. Plants are ordered by their prices. You want to find a plant that is close to but lower than your budget.

Given the root of the BST `inventory` and an integer `budget`, write a function `pick_plant()` that returns the plant with the highest price below `budget`. If no plant with a price strictly below `budget` exists, the function should return `None`.

```
class TreeNode:
    def __init__(self, key, val, left=None, right=None):
        self.key = key      # Plant price
        self.val = val      # Plant name
        self.left = left
        self.right = right

def pick_plant(inventory, budget):
    pass
```

Example Usage:

```
"""
               (50, "Fiddle Leaf Fig")
             /                        \
    (25, "Monstera")           (70, "Snake Plant")
       /        \                   /         \
(15, "Aloe")  (40, "Pothos")  (60, "Fern")  (80, "ZZ Plant")
"""

# Using build_tree() function at the top of page
values = [(50, "Fiddle Leaf Fig"), (25, "Monstera"), (70, "Snake Plant"), (15, "Aloe"),
            (40, "Pothos"), (60, "Fern"), (80, "ZZ Plant")]
inventory = build_tree(values)

print(pick_plant(inventory, 50))
print(pick_plant(inventory, 25))
print(pick_plant(inventory, 15))
```

Example Output:

```
Pothos
Aloe
None
```

▶ 💡 **Hint: Inorder Predecessor**

Close Section

▼ Problem 7: Remove Plant

A plant in your houseplant collection has become infested with aphids, and unfortunately you need to throw it out. Given the root of a BST `collection` where each node represents a plant in your collection, and a plant `name`, remove the plant node with value `name` from the collection. Return the root of the modified collection. Plants are organized alphabetically in the tree by value.

If the node with `name` has two children in the tree, replace it with its **inorder predecessor** (rightmost node in its left subtree). You do not need to maintain a balanced tree.

Pseudocode has been provided for you.

Evaluate the time complexity of your function. Define your variables and provide a rationale for why you believe your solution has the stated time complexity. Assume the input tree is balanced when calculating time complexity.

```
class TreeNode:
    def __init__(self, value, left=None, right=None):
        self.val = value
        self.left = left
        self.right = right

def remove_plant(collection, name):
    # Find the node to remove
    # If the node has no children
        # Remove the node by setting parent pointer to None
    # If the node has one child
        # Replace the node with its child
    # If the node has two children
        # Find the inorder predecessor
        # Replace the node's value with inorder predecessor value
        # Remove inorder predecessor
    # Return root of updated tree
    pass
```

Example Usage:

```
"""
              Money Tree
             /          \
          Hoya          Pilea
             \          /   \
             Ivy    Orchid  ZZ Plant
"""
```

```
# Using build_tree() function at the top of page
values = ["Money Tree", "Hoya", "Pilea", None, "Ivy", "Orchid", "ZZ Plant"]
collection = build_tree(values)

# Using print_tree() function at the top of page
print_tree(remove_plant(collection, "Pilea"))
```

Example Output:

```
['Money Tree', 'Hoya', 'Orchid', None, 'Ivy', None, 'ZZ Plant']

Explanation:
The resulting tree structure:
             Money Tree
            /          \
          Hoya        Orchid
             \           \
              Ivy       ZZ Plant
```

Close Section

Problem Set Version 2

- ▶ Problem 1: Find Lonely Cichlids
- ▶ Problem 2: Searching Ariel's Treasures
- ▶ Problem 3: Add New Treasure to Collection
- ▶ Problem 4: Sorting Pearls by Size
- ▶ Problem 5: Smallest Pearl Above Minimum Size
- ▶ Problem 6: Remove Invasive Species
- ▶ Problem 7: Minimum Difference in Pearl Size